

learning

Contract — Lessons, Materials, Assessment

All six routes sit behind the same publication decision and key on the opaque X-Session header.

Method	Route
GET	/api/areas/{area}/lessons
GET	/api/areas/{area}/lessons/{lesson}
POST	/api/areas/{area}/lessons/{lesson}/state
GET	/api/areas/{area}/materials (?kind= validated)
GET	/api/areas/{area}/assessment
POST	/api/areas/{area}/assessment/submit

Session

One channel: the header. The established interviewing module accepts a session three ways — body, query parameter, header. Three places to supply it are three places for them to disagree.

Materials

?kind= is validated against the closed vocabulary. **An unknown kind is a 400**, not an empty list — an empty list reads as “there are none”, which is a different and false statement.

A video material carries chapter_id, start_millis, end_millis, length_millis, transcript_anchor, chapter_slug, and an href built to [time-links.md](#).

Assessment — the non-disclosure guarantee

While an assessment is unavailable, the response carries **zero** question prompts, **zero** options and **zero** correct answers, and states what remains.

Enforced by construction, not convention. The domain `Question` type carries `correct_choices/explanation` with `omitempty`, so marshalling it directly would ship the key the moment a question had one. The taking projection is built as a map with **no answer field at all** — so no future upstream field can leak through it either.

Keys and explanations appear **only in the submit result**, where they are feedback.

Answers are identified **by ID, never by index**.

Availability

Decided by the kit's `AvailabilityOf`. Submission goes through `Submit`, never the ungated `Grade`. The consumer contains **no second availability test, no second scoring routine and no second answer comparison**.

Asserted by

`verify-assessment-gate.sh (0, 9 passed) · prove-assessment-gate.sh (0, 17 passed / 17 mutations) · a Go leak check that fails if correct_choices or explanation appears on the taking path.`

The load-bearing mutation is B4/M7 — “the gate opens as soon as *any* lesson completes”. It passes B1, B2, B3, B5 and B7; only B4 catches it. This is why the fixture's required set must be a **proper subset** of its lessons: a fixture where they coincide lets the defect through while looking thorough.

Vacuity refusals **B9/B10**: no lesson outside the required set, or a test gated on nothing, exit 2.

Fixtures

platform/qa/fixtures/learning/ — bicycle maintenance, **every byte invented**. Gates run against a gate-owned server built from source on a kernel-assigned free port. No private content reaches any gate log; the running container is never touched.